Software Configuration Steps Document

A Blockchain-Integrated Hybrid Cryptographic Framework for Secure and Tamper-Resistant Digital Forensic Evidence Storage System

Project	Digital Forensic Evidence Storage System
Department	CSE (Artificial Intelligence & Machine Learning)
Institution	Srinivasa Ramanujan Institute of Technology (SRIT)
Guide	Mr. K. Kondanna, M.Tech., (Ph.D.), Assistant Professor
Version	1.0
Date	April 2026

1. Overview

This document provides complete software configuration steps for setting up, deploying, and running the Blockchain-Integrated Hybrid Cryptographic Framework for Secure and Tamper-Resistant Digital Forensic Evidence Storage System. It covers environment setup, dependency installation, database configuration, blockchain network setup, and application launch procedures.

> **i** This system integrates Django (Python), AES-256/ECC hybrid encryption, blockchain audit logging (Hyperledger/Ethereum), and Multi-Factor Authentication (MFA) with Role-Based Access Control (RBAC).

2. System Requirements

2.1 Hardware Requirements

#	Component	Details
1	**Processor**	Intel Core i5 or higher (64-bit)
2	**RAM**	Minimum 8 GB (16 GB recommended for blockchain node)
3	**Storage**	Minimum 20 GB free disk space (SSD preferred)
4	**Network**	Stable internet connection (for blockchain sync and email/OTP services)
5	**Display**	1366 x 768 resolution or higher

2.2 Software Requirements

#	Component	Details
1	**Operating System**	Windows 10/11 (64-bit) or Ubuntu 20.04+
2	**Python**	Version 3.9 or 3.10
3	**Django**	Version 4.x (Web Framework)
4	**Node.js**	Version 16+ (for Ethereum/Hardhat tools)
5	**Database**	SQLite (development) / PostgreSQL 13+ (production)
6	**Blockchain**	Hyperledger Fabric or Ganache (Ethereum local testnet)
7	**Browser**	Google Chrome / Firefox (latest version)

#	Component	Details
8	**IDE**	VS Code or PyCharm
9	**Email Service**	Gmail SMTP or any SMTP provider (for OTP delivery)

3. Environment Setup

3.1 Install Python

1. Download Python 3.9 or 3.10 from https://www.python.org/downloads/
2. During installation, check 'Add Python to PATH'
3. Verify installation by opening a terminal and running:

```
python --version
pip --version
```

3.2 Install Node.js (Required for Blockchain Tools)

4. Download Node.js LTS from https://nodejs.org/
5. Install with default options.
6. Verify installation:

```
node --version
npm --version
```

3.3 Install Git

7. Download Git from https://git-scm.com/downloads
8. Install with default options.
9. Verify:

```
git --version
```

4. Project Setup

4.1 Clone or Extract the Project

If using Git, clone the repository:

```
git clone https://github.com/your-repo/forensic-evidence-system.git
cd forensic-evidence-system
```

Or extract the project ZIP archive and open a terminal in the project root folder.

4.2 Create a Virtual Environment

It is strongly recommended to use a virtual environment to isolate project dependencies.

```
# Windows
python -m venv venv
venv\Scripts\activate

# Linux / macOS
python3 -m venv venv
source venv/bin/activate
```

4.3 Install Python Dependencies

With the virtual environment active, install all required libraries:

```
pip install -r requirements.txt
```

If requirements.txt is not available, install packages individually:

```
pip install django==4.2
pip install pyotp
pip install pyjwt
pip install pycryptodome
pip install cryptography
pip install web3
pip install psycopg2-binary
pip install pillow
pip install django-environ
```

> **i** Ensure the virtual environment is always activated before running any project commands. You will see (venv) at the start of your terminal prompt when it is active.

5. Database Configuration

5.1 SQLite (Default — Development)

SQLite is preconfigured and requires no additional setup. The database file will be created automatically when migrations are applied.

5.2 PostgreSQL (Production)

10. Install PostgreSQL 13+ from https://www.postgresql.org/download/

11. Create a new database and user:

```
-- Open PostgreSQL shell (psql)
CREATE DATABASE forensic_db;
CREATE USER forensic_user WITH PASSWORD 'your_secure_password';
GRANT ALL PRIVILEGES ON DATABASE forensic_db TO forensic_user;
```

12. Update the DATABASE section in settings.py (or .env file):

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'forensic_db',
        'USER': 'forensic_user',
        'PASSWORD': 'your_secure_password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

5.3 Apply Migrations

```
python manage.py makemigrations
python manage.py migrate
```

> **i** Run migrations every time models are changed to keep the database schema in sync.

6. Environment Variables (.env Configuration)

Create a .env file in the project root directory and configure the following variables:

```
# Django
SECRET_KEY=your-very-secret-django-key
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

# Database (PostgreSQL)
DB_NAME=forensic_db
DB_USER=forensic_user
DB_PASSWORD=your_secure_password
DB_HOST=localhost
DB_PORT=5432

# Email (SMTP for OTP delivery)
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=your_email@gmail.com
EMAIL_HOST_PASSWORD=your_app_password
```

```
# JWT
JWT_SECRET=your-jwt-secret-key
JWT_EXPIRY_MINUTES=60

# Blockchain
BLOCKCHAIN_NODE_URL=http://127.0.0.1:8545
CONTRACT_ADDRESS=0xYourDeployedContractAddress
```

> **i** Never commit the .env file to version control. Add it to .gitignore to keep secrets safe.

7. Blockchain Configuration

7.1 Option A — Ethereum Local Testnet (Ganache)

13. Install Ganache CLI:

```
npm install -g ganache
```

14. Start the local blockchain:

```
ganache --port 8545 --deterministic
```

15. Install Hardhat for smart contract deployment:

```
npm install --save-dev hardhat
```

16. Deploy the audit logging smart contract:

```
npx hardhat compile
npx hardhat run scripts/deploy.js --network localhost
```

17. Copy the deployed contract address and paste it into CONTRACT_ADDRESS in your .env file.

7.2 Option B — Hyperledger Fabric

18. Install Docker and Docker Compose from https://docs.docker.com/
19. Clone the Hyperledger Fabric samples:

```
git clone https://github.com/hyperledger/fabric-samples.git
```

20. Start the test network:

```
cd fabric-samples/test-network
./network.sh up createChannel -c forensicchannel
```

21. Deploy the chaincode (smart contract):

```
./network.sh deployCC -ccn forensiclog -ccp ../chaincode/forensiclog -ccl python
```

> **i** For academic/testing purposes, Ganache (Ethereum local testnet) is simpler to configure. Use Hyperledger Fabric for production-grade deployments.

8. Email & MFA (Google Authenticator) Configuration

8.1 Gmail SMTP Setup

22. Log in to your Gmail account.
23. Go to Google Account > Security > 2-Step Verification and enable it.
24. Generate an App Password: Google Account > Security > App Passwords.
25. Select 'Mail' as the app and 'Windows Computer' as the device, then generate.
26. Copy the 16-character app password into EMAIL_HOST_PASSWORD in your .env file.

8.2 MFA QR Code Setup

The system auto-generates QR codes using PyOTP on user registration. Users must:

- Download Google Authenticator on their mobile device (Android / iOS).
- Scan the QR code sent to their registered email.
- Enter the 6-digit TOTP code during each login session.

To test QR generation manually in Django shell:

```
python manage.py shell
>>> import pyotp
>>> secret = pyotp.random_base32()
>>> totp = pyotp.TOTP(secret)
>>> print(totp.provisioning_uri('user@example.com', issuer_name='ForensicSystem'))
```

9. Create Admin Superuser & Run the Application

9.1 Create Superuser (Admin Account)

```
python manage.py createsuperuser
# Enter username, email, and password when prompted
```

9.2 Collect Static Files

```
python manage.py collectstatic
```

9.3 Run the Development Server

```
python manage.py runserver
```

Open a browser and navigate to: http://127.0.0.1:8000

9.4 Access the Admin Panel

Navigate to: http://127.0.0.1:8000/admin and log in with the superuser credentials created above.

> **i** Use the Admin Panel to approve new user registrations, assign roles (Investigator, Analyst, Legal Officer), and manage access requests.

10. System Module Verification Steps

After the server is running, verify each module using the following steps:

1	**User Registration**	Navigate to /register. Fill in name, email, and password. Check email inbox for QR code.
2	**MFA Setup**	Scan the received QR code using Google Authenticator app.
3	**Admin Approval**	Admin logs in to /admin panel, approves or rejects pending user accounts.
4	**User Login**	Navigate to /login. Enter credentials + TOTP code from Google Authenticator.
5	**Evidence Upload**	Admin navigates to /upload. Selects a file (PDF, DOC, MP4, etc.) and clicks Upload & Encrypt.
6	**Encryption Verify**	After upload, confirm AES-256 encrypted fragments appear in the admin Documents panel.
7	**Access Request**	User navigates to /documents, selects a file, and clicks Request Key.
8	**Admin Key Approval**	Admin navigates to /key-requests and approves the user's request.
9	**Key Delivery**	User receives a one-time decryption key via email.
10	**File Decryption**	User navigates to /view-document, selects the file, enters the key, and views the document.
11	**Blockchain Log**	Admin navigates to /blockchain-log to verify all activities are recorded immutably.
12	**Performance Metrics**	Admin navigates to /performance to review encryption/decryption timing data.

11. Common Issues & Troubleshooting

#	Component	Details
1	**ModuleNotFoundError**	Ensure virtual environment is activated and run pip install -r requirements.txt
2	**DB Migration Error**	Delete db.sqlite3 and migrations folder (except __init__.py), then re-run makemigrations and migrate
3	**Email Not Sent**	Check .env SMTP credentials. Ensure Gmail App Password (not account password) is used
4	**Blockchain Connection Error**	Ensure Ganache is running on port 8545. Verify BLOCKCHAIN_NODE_URL in .env
5	**OTP Invalid**	Verify system clock is synchronized (TOTP is time-based). Regenerate QR code if needed
6	**Port 8000 in Use**	Run: python manage.py runserver 8080 to use a different port
7	**Static Files Not Loading**	Run: python manage.py collectstatic and refresh the browser
8	**Admin Page 403 Forbidden**	Ensure the logged-in user has is_superuser = True in the database

12. Technology Stack Summary

#	Component	Details
1	**Backend**	Django 4.x (Python 3.9+)
2	**Frontend**	HTML5, CSS3, Bootstrap 5, JavaScript
3	**Authentication**	PyOTP (TOTP/MFA), PyJWT (JSON Web Tokens)
4	**Encryption**	PyCryptodome (AES-256-GCM), Cryptography library (ECC)
5	**Integrity**	hashlib (SHA-256), HMAC
6	**Blockchain**	Web3.py, Ganache / Hyperledger Fabric
7	**Database**	SQLite (dev), PostgreSQL (production)
8	**IDE**	VS Code / PyCharm
9	**Version Control**	Git